Rust 代码知识图谱需求清单 · 检视报告

1. 总体评价

1.1 优点（值得保留的强项）

强项	说明
生命周期覆盖完整	建图 → 持久化 → 重启加载 → 查询 → 增量更新 → 失败恢复 → 健康检查，形成闭环；比大多数同类工具的需求清单更完整。
状态与快照设计专业	building/complete/partial/failed/stale 状态机、"失败保留旧成功快照"、"原子发布新快照"等约束，直接命中知识图谱工具最易出错的地方。
Rust 语义贴合度高	Workspace/Crate/target、features/cfg 条件编译、Trait/Impl/REALIZES 方法映射、unsafe 块、宏与生成来源（GENERATED_FROM）、async（匿名 Future）、泛型/关联类型，均有专门条目，不是通用代码图谱能覆盖的。
不确定结果建模好	"未解析 Site / 外部 / 范围外 / 候选目标 / 声明锚点"等状态贯穿 A/B/C，避免把不确定解析当作事实，这是此类工具最常见的失败点。
查询边界与错误语义清晰	B 表区分参数错误 / 空集 / 歧义 / 能力未启用 / 截断 / 部分结果，且 B-30 要求 CLI 与 MCP 同源共享语义，工程上非常正确。

1.2 主要问题（概要 · 大方向）

> 本节整合需求大方向问题（功能/非功能边界、FFI、需求与设计分离、分层与优先级、双向关系、文档结构）。

1. **范围过大、缺乏分层与优先级**：MIR 级过程分析（CFG/DFG、Statement/Expression/Pattern/BasicBlock）、文件簇、语义/关键词检索、proc-macro 展开、内联汇编等与核心能力并列，未做优先级分期，v1 几乎不可完成；且需求条目间存在**包含关系**（如"项目建图"A-2 即涵盖后续 A-3~6 等具体需求），应先分层再排优先级。前期建议优先 A 表建图相关需求与 C 表具体的点/边（图完善是根本），增量/查询/MCP/非功能性需求可随开发与使用迭代补入，仅保留少量基础查询用于验证建图。

2. **Schema 与需求不一致**：多个被 A/B 表引用并承诺的实体/关系（FileCluster、测试、属性关联边、宏调用→宏定义边、约束、外部 stub、配置视图）在 C 表 Schema 中缺失或建模不明。

3. **需求与设计混写**：Salsa、图谱数据库等具体技术方案被直接写进"需求说明"，未区分 what 与 how。建议需求与设计分离：设计想法记入其他文档，或单开一列记录初步设计方案。

4. **C 表关系存在双向冗余**：CALLS/CALLED_BY 等双向关系不增加实际信息、却增加维护成本与不一致风险；B 表"调用者/被调用者"两种查询应由同一条有向边 CALLS 的正/反向推导。

5. **非功能性需求缺失**：目前只列了功能需求，性能/二进制形态/并发/可视化/资源占用/安全/易用性等均未列入（鲁棒性相关已基本覆盖，常见错误状态均有覆盖）。性能/并发/资源上限、任务取消、增量触发方式（watch 模式）、源码指纹（Git/内容 hash）方案、Schema 版本迁移、图谱导出等需补齐。

6. **混合编程（FFI/shim）边界未定**：Rust 常与 C/C++ 等混编，FFI 与 shim 层是否纳入知识图谱、如何建模（对应 C 表外部块/外部项，见 C-M08）需明确。

7. **文档结构问题**：A 表前后两段列名不一致（具体需求→具体更新要求、输入→涉及的图谱信息），建议拆分为两张表或统一列名；前后半段大量重复（持久化/加载/失败恢复/全量回退被多行重复覆盖）。

8. **接口说明**：表B 提供了大量的查询功能，功能越多，用户使用该工具的学习成本就越高，因此要制定一个易于用户学习的，能够让用户快速找到并学会使用符合其查询需求的查询命令。

---

2. 需求不一致问题

编号	级别	位置	问题描述	建议
X-01	高	C 表 vs A-17 / B-31	A-17 明确要求"可查询 FileCluster 与 MEMBER_OF_CLUSTER"，B-31 要求文件簇列表/成员/跨簇关系；但 C 表 34 个节点、28 条关系中**没有 FileCluster 节点，也没有 MEMBER_OF_CLUSTER 及跨簇关系边**。	二选一：①在 C 表补齐 FileCluster 节点 + MEMBER_OF_CLUSTER/簇间关系边；②将文件簇明确降级为"派生视图"而非 Schema 实体，并在 A/B 中注明。
X-02	高	C 表 vs B-19	B-19 变更影响分析承诺"如图谱中存在可识别测试，返回有关系依据的候选测试"，但 C 表**既无"测试"节点，也无 #[test]/#[bench] 属性标记说明**，测试无从识别。	在 C 表为函数节点补充"测试标记"属性或独立 Test 标识；否则弱化 B-19 的"候选测试"承诺。
X-03	中	C 表 vs A-25 / B-10 / B-24	A-25 使用"Constraint"一词，B-10 要求按"约束"角色查询类型使用，但 C 表仅把泛型约束作为各节点**字符串属性**（如"函数定义"节点的"泛型约束"），无独立 Constraint 实体或关系边。约束若要可查询、可随签名增量更新，仅存文本属性难以满足。	明确约束的建模粒度：P0 用属性；若需结构化查询（P2）则补 Constraint 节点 + 关系（如 CONSTRAINED_BY）。
X-04	中	C-25 属性节点	属性节点注释要求"关联被标注的代码实体"，但 28 条关系边中**没有"实体↔属性"边**（如 HAS_ATTRIBUTE / ANNOTATED_BY），属性将成为孤立数据。	补充实体→属性关联边；#[cfg] 等条件编译属性的满足状态也要可查（对应 A-7）。
X-05	中	C-27 宏调用点	宏调用点注释要求"关联…宏定义"，但关系清单中只有 GENERATED_FROM（生成实体→宏调用点）和 TARGETS（语义偏向函数/方法/构造调用），**缺"宏调用点→宏定义"的显式边**。	补充 EXPANDS / INVOKES 边（宏调用点→宏定义），否则无法从调用反查定义。
X-06	中	C 表整体	**节点名全中文（函数定义、结构体…），关系名全英文（CONTAINS、IMPLEMENTS...）**，命名语言不一致。	统一为英文标识符（节点英文 ID + 中文注释），便于代码生成、持久化与跨工具引用。

编号	级别	位置	问题描述	建议
X-07	中	C-56 / C-57	同时存储 CALLS 与 CALLED_BY 并要求"保持一致"，存在双写不一致风险与额外写入量；且属**双向冗余**，不增加实际信息（用户已确认为方向性问题，见 1.2-4）。	只存一个方向（如 CALLS），被调用者/调用者分别由正向/反向查询推导；B 表"调用者/被调用者"两查询共用同一条边。
X-08	中	A-11 / A-34	"Salsa 负责计算复用，图谱数据库负责磁盘持久化"、"选取 Salsa 作为增量更新框架"把**实现方案写进需求**。	分层编写：需求列写"必须支持增量复用、可持久化、可跨进程读取"，实现方案（Salsa、具体 DB）作为技术决策记录单独说明。
X-09	中	A 表前后段	大量重复：A-10/11/12（持久化、重启加载、多快照）与 A-38（持久化与重启）重复；A-15、A-32、A-33（失败恢复/一致性与发布/重试回退）三行高度重叠；A-9（全量重建）与 A-33 部分重叠。	合并同类项，把"原子发布 + 失败保留旧快照 + 状态机"作为一条横切性非功能需求统一描述，避免需求漂移。

3. 表 A「图谱构建与管理」检视

3.1 问题（现有条目）

编号	级别	行	问题描述	建议
A-I01	中	A-4 / A-35	"生成或虚拟文件"纳入解析 + "捕获 build.rs/proc-macro 产物、include!": Rust 中 proc-macro 展开与 build 脚本产物解析是公认难点，若承诺"展开并分析生成代码的实体"成本极高。	明确支持程度：是"真正展开 proc-macro 并抽取生成实体"，还是"仅记录 GENERATED_FROM 来源、不深入分析生成体"。避免承诺超出可实现范围。
A-I02	中	A-7	"未求值或范围外代码不能默认为 active"——原则正确，但未定义"未知（unknown）状态如何产生、如何随配置变化翻转"。	**（这个可能更偏向于设计了）**补充 cfg 求值规则与 unknown 状态的判定/传播机制。
A-I03	低	A-16	健康检查中"重复限定名或孤立节点作为审查提示，不全部判为错误"——好；但未定义"建议处理动作"的产出物（自动修复 or 仅报告）。	明确健康检查只读、不自动修改图谱。

3.2 缺失项（应补充）

编号	级别	缺失需求	说明与建议
A-M01	高	**性能与规模目标**	全清单无任何关于建图规模（文件数/实体数上限）、并行解析、内存上限、构建耗时的目标或约束。大型 Rust 项目（数千文件、依赖图）下性能直接决定可用性。建议补充：规模目标、并行度、资源限制、进度可观测。

编号	级别	缺失需求	说明与建议
A-M02	中	**任务取消/暂停控制**	只有"重试/继续/全量重建"，无"取消进行中的构建/更新"需求。长时间建图需要可取消与干净的中断处理。
A-M03	中	**增量触发方式与更新频率**	A-18 后整段讲增量更新，但未说明触发机制与频率：用户修改代码后是**手动触发更新**，还是工具**自动识别并更新**（文件监听 watch / 查询时懒触发）？本地开发工具最常见诉求是 watch 自动更新；建议明确触发时机与频率约束。
A-M04	中	**源码指纹/版本识别方案**	A-9/10/12 反复使用"源码版本标识""判断配置变化"，但未定义指纹方案（git commit/hash、文件内容 hash、mtime+size）与指纹粒度。建议补一条"输入指纹"需求：文件级、算法稳定、纳入快照。
A-M05	中	**存储方案与 Schema 迁移**	A-11 只写"图谱数据库负责磁盘持久化"，未定义存储选型约束（嵌入式 vs 服务、单文件可移植性、备份）；全篇无"Schema 版本升级与图谱迁移"需求（A-12 只校验兼容性，未给不兼容时的迁移路径）。工具版本演进必然发生，需补迁移与兼容策略。
A-M06	低	**Cargo.lock 的输入地位**	A-28 提到"锁文件"变化触发重建，但 A-2/A-9 的输入与快照内容未把 Cargo.lock 列为固定输入，依赖版本可复现性无保障。
A-M07	低	**图谱导出/互通**	无导出（JSON/GraphML/CSV）需求。对知识图谱而言，导出便于审计、测试与外部消费。
A-M08	中	**Unsafe 分析边界**	C 表含 Unsafe 块节点与 NESTED_IN，B-25 提供 unsafe 定位查询，但 A 表没有对应构建期需求（何时、以何层级提取 unsafe 块，是否进入 MIR 验证）。
A-M09	中	**实体身份稳定性规则**	A-4/A-26 要求"同名短名称不得自动合并""不能让旧 ID 静默指向另一实体"，但未定义**实体 ID 的生成规则与稳定性**（如基于限定名+源码位置+配置的稳定 hash），这是增量更新与跨快照一致性的根基。
A-M10	中	**回调/高阶函数关系**	建图与 C 表均缺少"回调"关系：函数/闭包作为**实参被传入**另一函数（如 iter().map(f)、thread::spawn(f)）的关系未纳入语义关系构建与增量更新，B 表也无对应查询。建议在 A 表建图需求中补充"函数/闭包作为值传递/回调"的识别（与 C 表回调边配套，见 C-M13）。

3.3 非必要 / 可裁剪项（建议降级）

编号	级别	行	条目	理由与建议
A-N01	高	A-17 / A-37	文件簇构建与管理（含增量重算）	属"锦上添花"的派生分析：需聚类算法、参数/版本管理、重叠策略、重算机制，且与核心实体/关系查询解耦。建议划为 P2；若保留，先补齐 C 表 Schema（见 X-01）。
A-N02	中	A-36	计算图｜循环与递归处理	本质是内部正确性/终止性约束，不是用户可见功能。建议并入"增量更新框架/一致性"作为约束条件，而非独立需求行。

编号	级别	行	条目	理由与建议
A-N03	中	A-9	全量重建（独立成行）	是既有能力的组合（新快照 + 可选缓存复用），可与 A-33"全量回退"合并表达，减少重复。
A-N04	低	A-20~29	增量变更场景拆行过细	10 行变更场景中 A-23（改函数体）与 A-20/21 高度相关、A-29（范围变化）与 A-6+A-27 重叠。建议按"实体级/结构级/配置级"分组合并，减负评审与实现。

4. 表 B「图谱查询与使用」检视

4.1 问题（现有条目）

> 检视说明：B 表末两列（GitNexus / CodeGraph 关键字）为能力映射参考，本次一并检视（编号 B-I05 起）。以下意见基于两工具公开能力与命名习惯，个别无法联网核实的条目标注"存疑"。

编号	级别	行	问题描述	建议
B-I01	中	B-19	"候选测试"承诺依赖可识别测试实体，但 C 表无测试建模（见 X-02）。	补 Schema 或弱化表述为"若图谱已标注 #[test] 并建立其依赖，则返回候选测试"。
B-I02	低	B-3 / B-4	B-3 模糊查询的"包含匹配"与 B-4 关键词检索在名称层面重叠，边界不清。	明确 B-3 只做结构化名称匹配（前缀/包含/大小写），B-4 负责跨字段（名称/签名/注释/源码）全文/语义检索。
B-I03	低	B-6	"按 C 类范围提供局部绑定等信息"依赖 C 表 Statement/Expression/Pattern 节点；若这些 MIR 级节点按建议延后（见 C-N01），此条款应同步标注为可选。	在需求中注明该子句为"可选深度能力，未启用时明确返回未启用状态"。
B-I04	低	B-13	A→B 调用链默认只查确定调用、纳入候选时整条路径标不确定性——写法很好；但未定义"最短路径"算法语义（BFS 按调用边层级）与路径去重。	补充路径去重与环处理的具体语义，避免实现歧义。
B-I05	中	B 表末两列（关键字）	关键字列用于映射 GitNexus/CodeGraph 能力，但若干条目与真实工具能力不符或存疑，且未标注命名域（CLI/MCP/资源），易误导"功能对齐"的承诺。	关键字列仅作参考，统一标注 [CLI]/[MCP]/[资源] 前缀，注明"为参考映射、不承诺功能对齐"；存疑条目核实或删除（见 B-I06~B-I14）。

编号	级别	行	问题描述	建议
B-I06	中	B-4 CodeGraph 关键字	关键词/任务描述检索映射到 explore/codegraph_explore 不准确：CodeGraph 的 explore 是"符号间路径/关联探索"（与 B-13 同款语义），语义检索应为 query/codegraph_search（恰为 B-3 列所写）；B-3 与 B-4 两行的 CodeGraph 关键字疑似对调/混用。	更正映射：B-4 用 query/codegraph_search；B-3 模糊查询在 CodeGraph 中无真正对应（query 为语义检索），注明近似程度。
B-I07	中	B-27 GitNexus 关键字	pdg_query(mode=controls|flows)、explain、analyze --pdg 疑为虚构或超出 GitNexus 公开能力（PDG/过程依赖图查询），无依据（存疑）。	与真实 GitNexus 能力核对；无法证实时删除，仅保留可证实的 cypher。
B-I08	低	B-28 GitNexus 关键字	CLI list 命令名不准确（GitNexus 应为 list-repos）；MCP 侧漏 status 工具；"repo context 资源"表述含糊。	统一为 list-repos（CLI）/list_repos（MCP），补 MCP status，资源 URI 写明 gitnexus://repo/{name}/context。
B-I09	低	B-29 GitNexus 关键字	漏列 MCP schema 工具；资源 URI 格式需统一。	补 MCP schema；资源 URI 统一为 gitnexus://repo/{name}/schema。
B-I10	低	B-30 GitNexus 关键字	MCP 侧只列了 detect_changes，与"统一提供 CLI 与 MCP"的语义不符。	列全 MCP 工具族（query/context/impact/trace/detect_changes/list_repos/status/schema/cypher 等）与 CLI 一一对应。
B-I11	低	B-13 GitNexus 关键字	trace(from_uid,to_uid,maxDepth) 参数名 maxDepth 与真实工具命名（depth）不一致（存疑）。	统一参数命名并注明 trace 的"路径/影响"语义与默认深度。
B-I12	低	B-31 GitNexus 关键字	cluster/{clusterName} 缺完整资源前缀；GitNexus 是否提供 clusters 资源需核实（存疑）。	统一为 gitnexus://repo/{name}/clusters/{clusterName} 并核实；未证实时可仅保留 cypher 表达。
B-I13	低	B-18 CodeGraph 关键字	affected 命令是否存在存疑（作者已自注"变更相关测试，非完整差异"），与"源码与版本变更检查"语义不完全对应。	核实 CodeGraph 命令集；若无则删除并注明 CodeGraph 无对应能力。
B-I14	低	关键字列整体	两列命名域混用：GitNexus 列 MCP 工具用 snake_case（detect_changes/list_repos），CodeGraph 列统一 codegraph_* 前缀；同一行的 CLI 与 MCP 命名未区分。	统一标注 [CLI]/[MCP]/[资源] 前缀，明确两列仅为参考映射。

4.2 缺失项（应补充）

编号	级别	缺失需求	说明与建议
B-M01	中	**条件编译 / feature 视图查询**	A-7 要求实体/关系带"配置/视图归属"，但 B 表没有"某符号在哪些 cfg/feature 下生效""当前视图下哪些符号被条件编译排除""某 feature 的依赖与影响范围"等查询。建议在"Rust 语义查询"或"图谱信息查询"下补充配置视图查询。
B-M02	中	**公共 API / 导出符号清单**	无"列出某 Crate/模块的公开 API（pub 符号、重导出、Trait 约束摘要）"查询。B-8 模块导入与重导出只覆盖重导出方向，不构成 API 面查询；这是理解库边界的最常见需求。
B-M03	低	**未使用/死代码查询**	基于 REFERS_TO/USES 可推导"零引用定义"，对重构与清理高价值、低成本，建议作为可选查询（不承诺可达性/数据流证明，避免与 B-27 深层分析混淆）。
B-M04	中	**快照间差异查询（面向已建图谱）**	B-18 变更检查是"输入侧"（当前源码 vs 基线）；缺"图谱快照 A vs 快照 B"的差异查询，便于两个已建图谱的回归对比。建议补充或明确其归口。
B-M05	低	**整图统计与热点分析**	B-29 有组合查询/聚合统计，但无"按节点/边类型计数、高扇入/扇出符号、未解析热点"等面向理解的聚合视图。属锦上添花，建议 P2。
B-M06	低	**模块级符号大纲**	B-16 是"文件符号大纲"（按文件）；跨文件模块（mod 分散多个文件）缺少"模块级全部声明"查询。Rust 模块常跨文件，建议补充模块级浏览。

4.3 非必要 / 可裁剪项（建议降级）

编号	级别	行	条目	理由与建议
B-N01	中	B-4	关键词与任务描述检索	语义/全文检索涉及索引、排序（甚至 embedding），实现成本高、结果解释性弱。作为"发现入口"有价值，建议 P2 或先做"名称+注释"轻量检索（复用 B-3），暂不承诺语义检索。
B-N02	中	B-27	可选深层分析结果读取（CFG/DFG）	依赖 C 表 MIR 级节点（BasicBlock、CONTROL_FLOW、DATA_FLOW、Statement/Expression），与核心知识图谱价值距离较远、成本高。建议整组划为 P2；该条本身"未启用即明确状态"的写法良好，仅延后实现。
B-N03	低	B-31	文件簇查询	随 A-17 文件簇降级一并延后（见 A-N01）；若实现需与 C 表 Schema 对齐（见 X-01）。

编号	级别	行	条目	理由与建议
B-N04	中	B-3	模糊查询	用户意见：GitNexus 与 CodeGraph 均未能做好模糊查询，若要做需从实现原理（ngram 索引、fuzzy 匹配引擎）重新考量。建议降低优先级（P2 之后），P0 阶段以精确匹配 + 通配符近似替代。

5. 表 C「图谱Schema」检视

5.1 问题（现有条目）

编号	级别	位置	问题描述	建议
C-I01	中	信息获取层级列	大量节点标注 HIR，但"调用目标解析、TYPE_REF、REFERS_TO 目标解析"实际需要名称解析+类型推断+方法解析，已超出纯 HIR，接近 rust-analyzer 语义层。	将"HIR/MIR"标注改为"信息来源层级 + 语义分析层级"（如 HIR + rust-analyzer 语义解析），避免误导实现者。
C-I02	中	C-33 Type 节点	注释称"类型既可作为各节点属性，也可保留为 Type 节点"——**双轨制未定义何时用哪种**，易同一信息两处存、查询不一致。	建议类型作为属性，TYPE_REF关系只用于 typedef。
C-I03	低	C-17/22/23	关联常量、全局常量、静态变量语义接近（同为 const/static 值项，仅归属不同），容易混淆。	在注释中显式区分"顶层 const/static vs 关联 const"，或合并为"常量"节点+归属属性。
C-I04	低	C-36 CONTAINS 等	关系注释只写了"示例"，未说明**关系是否带方向、是否可带属性（如 CALLS 关联调用点、TARGETS 的目标角色）**的统一规则。	补充"关系属性规范"：每条边可携带的属性（位置、状态、角色、版本等），与 A-5"保留关系证据"对齐。
C-I05	低	C-38 DEPENDS_ON	只记录 Crate→Crate 依赖，未覆盖"依赖的具体 target/feature 条件"与"依赖是否 external/范围外"的统一标记（见 C-M04）。	明确 DEPENDS_ON 的属性集合（版本、类别 normal/dev/build、生效条件、外部标记）。
C-I06	低	C-54 NESTED_IN	注释只描述 unsafe 块嵌套，但关系名未表达"unsafe"，语义与命名不匹配。	改名如 UNSAFE_NESTED_IN / NESTED_UNSAFE_IN；若想通用化，则需在注释中明确非 unsafe 场景并加关系类型属性。
C-I07	低	C-61 USES	关系名 USES 易被误认为 use 导入语句（IMPORTS 场景）。	建议改名（如 VALUE_USES / READS），与 IMPORTS/use 语义区分。

编号	级别	位置	问题描述	建议
C-I08	中	C-26/27 调用点 Site / 宏调用点 / TARGETS	对**每一处调用点都建节点**，点的数量会随代码量线性甚至超线性增长（大型项目调用点极多），存储与更新成本高。GitNexus/CodeGraph 用"边"表达调用：调用者与被调用者两个点 + 一条有向边即可。	将"调用点"从独立节点改为**调用边上的属性/证据**（位置、次数、角色、源码 span），既满足 B-11/12"每次调用可定位到文件行列"，又避免节点爆炸；宏调用点同理（边属性 + GENERATED_FROM）。
C-I09	中	C-04 File 节点	未定义 File 节点覆盖的**文件类别范围**：项目中的 .txt/.xlsx/.pptx/.mp3/无后缀文件是否都要建节点?	明确文件类别策略：代码与相关文件（.rs、Cargo.toml、build.rs、生成文件、include 目标）建节点；非代码文件默认不建节点或仅记录元数据（对应 A-6 范围/排除，避免图谱被非代码文件淹没）。
C-I10	低	C-29/30/32 种类属性	Pattern/Expression/Statement 节点的"种类"枚举需覆盖 Rust 1.89 Stable 已稳定的语言形式（如 let 链、deref pattern、async 闭包、C-unwind ABI 等），不能按早期版本枚举固化。	将种类枚举定义为可扩展集合，并在 Schema 中登记 1.89 Stable 新增形式（版本号以官方 release notes 为准）。

5.2 缺失项（应补充）

编号	级别	缺失项	说明与建议	建议
C-M01	高	**FileCluster 节点 / MEMBER_OF_CLUSTER 及跨簇关系**	A-17、B-31 明确引用，但 C 表完全缺失（见 X-01）。	
C-M02	中	**测试实体/标记**	B-19 需要（见 X-02）。建议在函数节点上加 is_test 属性或独立 Test 标识。	
C-M03	中	**属性关联边、宏定义关联边**	实体↔属性、宏调用点→宏定义均无对应边（见 X-04 / X-05）。	
C-M04	中	**外部依赖 / 范围外状态建模**	A-8 要求"建立 external stub、外部节点标记 external/范围外"，但 C 表未定义 ExternalCrate/Stub 节点，也未说明"范围外"状态字段的统一表达。	明确：外部依赖用 Crate 节点+状态属性，还是独立节点类型；并定义 DEPENDS_ON/TARGETS 上的 external 标记。
C-M05	中	**配置/视图（Config/View）维度**	A-7/9 要求实体/关系带"配置/视图归属"，C 表无 Config 节点，也未说明归属是节点/边属性还是独立维度。	明确"事实视图"实现方式（属性 vs 独立节点 vs 多版本边），直接决定增量与查询实现复杂度。

编号	级别	缺失项	说明与建议	建议
C-M06	低	**宏展开产物的实体化边界**	A-35 要求宏展开产物参与更新，C 表用 GENERATED_FROM 表达来源，但未规定"展开产物是否作为独立实体入库、其可查询性"。	补充展开产物的实体化策略（实体入库 or 仅来源记录）。
C-M07	低	**局部绑定作用域粒度**	C 有 局部变量 / Statement / Pattern / DEFINES / USES / ASSIGNS，但未说明粒度是"函数级"还是"块级"。	明确作用域粒度，避免与 MIR 级节点（见 C-N01）一起失控。

Rust 1.89 Stable 一等公民覆盖检视：以 Rust 1.89 Stable（含 Edition 2024 已稳定特性）的"一等公民"为基准逐项比对 C 表：类型级（基元/引用/指针/fn 指针/dyn/impl Trait/数组/切片/元组）已由 Type 节点"类型类别"承载；表达式/模式/语句级由 Expression/Pattern/Statement 节点承载；async fn/async 块由"匿名 Future"承载；const/static 由全局常量/静态变量/关联常量节点承载；模块/函数/结构体/枚举/联合体/Trait/Impl/类型别名/宏定义/属性等 items 均已覆盖。**未覆盖或建模不明确的一等公民如下（C-M08 起）**：

编号	级别	缺失项	说明与建议	建议
C-M08	高	**外部块与外部项（extern block / extern fn / extern static）**	Rust 一等公民（FFI 入口，extern "C" { } 及其中 extern fn/static/类型），当前 C 表无对应节点（"函数声明"仅以属性提及 ABI，未覆盖 extern block 本身与外部静态项）。直接关系到大方向"混合编程/FFI 边界"（见 1.2-6）。	增加 ExternBlock 节点（记录 ABI、来源），并明确 extern fn（归"函数声明"+ABI）与 extern static（外部静态项）的建模；同步在 A 表建图需求中明确 FFI/shim 层是否纳入。
C-M09	中	**Use/导入声明（use 声明项）**	use 是 Rust 一等公民 item，当前仅以 IMPORTS 边表达，无"导入声明"实体；B-8 需按导入保留文本、位置、别名与已解析目标。	增加 Use 节点（或"导入项"实体：路径文本、别名、可见性、位置、目标），与 IMPORTS/RE-EXPORTS 边配套。
C-M10	中	**泛型参数（类型/生命周期/常量参数）**	泛型参数（含生命周期、const 泛型）是 Rust 一等公民，当前仅作为节点属性，生命周期参数完全未建模；与 X-03（约束建模）同源。	增加泛型参数建模（至少生命周期参数 + 类型参数 + const 参数的结构化表示），支持 A-22/B-24 的泛型查询与增量更新。
C-M11	低	**proc-macro 声明（derive / attribute / function-like）**	三类 proc-macro 是 Rust 一等公民，且定义在独立 crate；"宏定义"节点需明确其宏种类覆盖 proc-macro，并说明跨 crate 的宏定义定位。	在"宏定义"节点上扩展宏种类枚举（macro_rules!/derive/attribute/function-like proc-macro），注明 proc-macro 定义位置可能在外部 crate。
C-M12	低	**async 闭包 / async 块标记**	Rust 1.85（Edition 2024）稳定 async 闭包，属一等公民表达式；当前"闭包"节点未记录 async 标记（"匿名 Future"仅提及 async 闭包为其生产者）。	在"闭包"节点补充 async/async move 标记；"匿名 Future"注明其由 async fn/async 块/async 闭包产生。

编号	级别	缺失项	说明与建议	建议
C-M13	中	**回调/高阶函数关系（CALLBACK / PASSED_AS_CALLBACK）**	函数/闭包作为实参被传入的关系缺失（如 iter().map(f)、thread::spawn(f)），A 表建图与 C 表均无对应边（关联 A-M10）。	增加回调/函数值传递边（如 PASSED_AS_CALLBACK / CALLBACK_TARGET），并纳入 B 表调用/引用查询（低优先级）。

5.3 非必要 / 可裁剪项（建议降级）

编号	级别	位置	条目	理由与建议
C-N01	高	C-31/29/30/32 + C-59~62	**MIR 级过程分析组**：BasicBlock、Statement、Expression、Pattern 节点 + CONTROL_FLOW、DATA_FLOW、DEFINES、USES、ASSIGNS 边	这些构成"程序切片/过程内分析"能力，工作量巨大（MIR 稳定解析 + 与增量框架集成），对"代码知识图谱"核心价值（结构/调用/类型/Trait）为增强项而非必需项。建议整体划为 P2 可选能力，与 B-27 对应；基础版先保证 实体+调用+引用+类型+Trait+宏来源。
C-N02	中	C-34 / C-63	匿名 Future、SPAWNS	async 语义有价值，但"匿名 Future 身份/输出类型/来源"与线程/任务启动关系属进阶能力。建议 P2；基础版保留 async fn/await 的调用关系即可。
C-N03	低	C-35	内联汇编 InlineAsm	使用面窄、依赖 rustc 细节，对多数项目价值低。建议 P3 或删除（可用 Attribute 节点记录 asm! 存在即可）。
C-N04	低	C-33	Type 节点（在属性双轨制下）	若 P0 阶段类型信息以节点属性承载，Type 独立节点可延后，待 TYPE_REF 结构化查询有需求时再引入。

---

6. 优先级与分期建议（P0 / P1 / P2 / P3）

> 原则：先做"可证明正确、可增量维护"的核心事实视图，再叠加高价值派生能力；凡依赖 MIR 级分析或聚类/检索算法的，一律延后。
> 按用户前期建议：优先 A 表建图相关需求与 C 表具体的点/边（图完善是根本），增量/查询/MCP/非功能性需求可随开发与使用迭代补入；仅保留少量基础查询用于验证建图正确性。

P0 —— MVP（核心图谱 + 基础查询）

- **A**：A-2~15（项目建图、Workspace/Crate 识别、实体提取、语义关系、范围与排除、条件编译配置、外部依赖边界、配置快照、全量重建、持久化、重启加载、多项目/多快照、构建摘要与状态、失败恢复、健康检查）。

· **B**：符号查询（精确 B-2、声明/定义 B-5、详情与成员 B-6）、依赖查询（B-7/8/9/10）、调用关系（B-11/12/13/14）、文件查询（B-15/16/17）、源码与上下文（B-20/21）、Rust 语义查询（B-22/23/24/25/26）、图谱信息查询（B-28/29）、对外接口（B-30）。（模糊查询 B-3 已按用户意见降级至 P2，见 B-N04）

· **C**：核心节点（Workspace/Crate/File/Module/函数定义与声明/方法/闭包/结构体/枚举/枚举元素/联合体/Trait/Impl/字段/关联常量/关联类型/类型别名/参数/局部变量/全局常量/静态变量/宏定义/属性/调用点 Site/宏调用点/Unsafe 块）+ 核心边（CONTAINS/LOCATED_IN/DEPENDS_ON/IMPORTS/REFERENCES_FILE/DECLARES/HAS_METHOD/HAS_MEMBER/HAS_PARAM/TYPE_REF/SELF_TYPE/IMPLEMENTS/REALIZES/INHERITS/REFERS_TO/INSTANTIATES/HAS_SITE/TARGETS/NESTED_IN/GENERATED_FROM/CALLS/CALLED_BY），并补齐 X-04/X-05 两条缺失边。

P1 —— 增量与维护闭环

· 增量更新框架（A-34，Salsa）与变更场景（A-20~29 分组后）、未变化结果复用（A-30）、受影响范围传播（A-31）、一致性与发布（A-32）、失败恢复重试（A-33）、外部输入/宏/生成代码（A-35）、计算图终止性（A-36 作为约束）。

· C 表一等公民补齐：外部块/外部项（C-M08）、use 导入声明（C-M09）、泛型参数（C-M10）、回调关系（C-M13）、proc-macro 声明（C-M11）、async 闭包标记（C-M12）。

· C 表去双向冗余：仅存 CALLS，被调用者由反向推导（X-07 / 1.2-4）。

· 补充横切性需求（A-M01~A-M10、B-M01~B-M06 中标记为高/中的项）。

· 变更查询（B-18/19，与 Schema 对齐后）。

P2 —— 高价值派生能力

· 文件簇（A-17/37、B-31，先补 C 表 Schema，见 X-01）。

· MIR 级过程分析组（C-N01 全部 + B-27）。

· 模糊查询（B-3）与关键词/任务描述检索（B-4，轻量版）——检索类能力需从实现原理重新考量（见 B-N04）。

· 匿名 Future / SPAWNS（C-N02）。

· 结构化约束 / Type 节点双轨统一（X-03、C-I02）。

P3 / 删除

· 内联汇编 InlineAsm（C-N03）。